

1 Solution — GIAM 1.4

Problem 8

1.1 Problem

Use the tables from the previous problem to make the following conversions: (a) 757_8 to binary; (b) 1007_8 to hexadecimal; (c) 100101010110_2 to octal; (d) 1111101000110101_2 to hexadecimal; (e) FEED_{16} to binary; (f) FFFFFF_{16} to octal.

1.2 The Method

Binary is the hub. Each **octal** digit is a **3**-bit block and each **hex** digit is a **4**-bit block (Problem 7). So every conversion is just *grouping*:

- to/from **octal** $\leftrightarrow$ binary: translate digits $\leftrightarrow$ **3**-bit blocks;
- to/from **hex** $\leftrightarrow$ binary: translate digits $\leftrightarrow$ **4**-bit blocks;
- **octal** $\leftrightarrow$ **hex**: route through binary — expand to bits, then *re-group* (**3** $\leftrightarrow$ **4**).

When grouping bits, count *from the right*, padding the leftmost group with leading zeros if needed.

1.3 The Six Conversions

(a) $757_8 \rightarrow$ **binary**. Replace each octal digit by its **3**-bit block: $7 \rightarrow 111$, $5 \rightarrow 101$, $7 \rightarrow 111$.

$$757_8 = \underbrace{111}_7 \underbrace{101}_5 \underbrace{111}_7 = \boxed{111101111_2}.$$

(b) $1007_8 \rightarrow$ **hex**. First expand to binary (**3**-bit blocks): $1\ 0\ 0\ 7 \rightarrow 001\ 000\ 000\ 111$. Then re-group into **4**-bit blocks from the right and read off hex:

$$1007_8 = 001000000111_2 = \underbrace{0010}_2 \underbrace{0000}_0 \underbrace{0111}_7 = \boxed{207_{16}}.$$

(c) $100101010110_2 \rightarrow$ **octal**. Group the bits into threes from the right:

$$\underbrace{100}_4 \underbrace{101}_5 \underbrace{010}_2 \underbrace{110}_6 = \boxed{4526_8}.$$

(d) $1111101000110101_2 \rightarrow \text{hex}$. Group into fours from the right:

$$\underbrace{1111}_F \underbrace{1010}_A \underbrace{0011}_3 \underbrace{0101}_5 = \boxed{\text{FA35}_{16}}.$$

(e) $\text{FEED}_{16} \rightarrow \text{binary}$. Replace each hex digit by its 4-bit block: $F \rightarrow 1111$, $E \rightarrow 1110$, $E \rightarrow 1110$, $D \rightarrow 1101$.

$$\text{FEED}_{16} = \underbrace{1111}_F \underbrace{1110}_E \underbrace{1110}_E \underbrace{1101}_D = \boxed{111111011101101_2}.$$

(f) $\text{FFFFFF}_{16} \rightarrow \text{octal}$. Each F is 1111, so FFFFFF_{16} is *twenty-four* 1's. Re-group those 24 bits into eight threes:

$$\text{FFFFFF}_{16} = \underbrace{1111 \cdots 1111}_{24 \text{ ones}}_2 = \underbrace{111}_7 \underbrace{111}_7 \cdots \underbrace{111}_7 (\times 8) = \boxed{77777777_8}.$$

1.4 The Grouping at Work

Binary is the hub: octal groups the bits by 3, hexadecimal groups them by 4.

(a) octal $\rightarrow$ binary (each digit = 3 bits)

$$757_8 = \begin{array}{|c|c|c|} \hline 111 & 101 & 111 \\ \hline \end{array} \quad \begin{array}{c} 7 \quad 5 \quad 7 \end{array} = 111101111_2$$

(e) hex $\rightarrow$ binary (each digit = 4 bits)

$$\text{FEED}_{16} = \begin{array}{|c|c|c|c|} \hline 1111 & 1110 & 1110 & 1101 \\ \hline \end{array} \quad \begin{array}{c} F \quad E \quad E \quad D \end{array} = 111111011101101_2$$

(f) hex $\rightarrow$ octal, routed through binary (regroup 4-bit blocks into 3-bit blocks)

$\text{FFFFFF}_{16} = \text{twenty-four 1's}$:

by 4 (hex): $1111 \cdot 1111 \cdot 1111 \cdot 1111 \cdot 1111 \cdot 1111 = F F F F F F$

by 3 (octal): $111 \cdot 111 \cdot 111 \cdot 111 \cdot 111 \cdot 111 \cdot 111 \cdot 111 = 7 7 7 7 7 7 7 7$

$$\boxed{\text{FFFFFF}_{16} = 77777777_8 = 2^{24} - 1}$$

Figure 1.1: (a) $757_8 \rightarrow$ each octal digit becomes a 3-bit block $\rightarrow 111101111_2$. (e) $\text{FEED}_{16} \rightarrow$ each hex digit becomes a 4-bit block $\rightarrow 111111011101101_2$. (f) FFFFFF_{16} is twenty-four 1's; grouped by 4 it reads FFFFFFFF, grouped by 3 it reads 77777777 — so $\text{FFFFFF}_{16} = 77777777_8 = 2^{24} - 1$.

1.5 Summary

Part	Given	Convert to	Answer
(a)	757_8	binary	111101111_2
(b)	1007_8	hex	207_{16}
(c)	100101010110_2	octal	4526_8
(d)	1111101000110101_2	hex	$FA35_{16}$
(e)	$FEED_{16}$	binary	111111011101101_2
(f)	$FFFFFF_{16}$	octal	7777777_8

Table 1.1

1.6 Remark — Why No Arithmetic Is Needed

Every conversion above used only table look-up and grouping — *no division, no multiplication, no carrying*. That is the whole payoff of binary being the common hub, and it works precisely because $8 = 2^3$ and $16 = 2^4$ are powers of 2: each octal or hex digit owns a self-contained block of bits, so digits never interact. Contrast this with the *repeated-division* method of Problem 4, which was needed only because decimal ($10 \neq 2^k$) shares no such clean bit-grouping with binary. For the power-of-two bases, conversion is pure clerical bookkeeping.

1.7 Remark — Padding from the Right

The rule “group from the right, pad the left” matters whenever the bit count is not a clean multiple of the block size. In (b) the binary string 001000000111 has 12 bits — already a multiple of both 3 and 4 — but in general one pads. For instance, to octal-convert 100101010110_2 (part (c), 12 bits, fine) versus a 10-bit string like 1011001110_2 : group as $1\ 011\ 001\ 110 \rightarrow 001\ 011\ 001\ 110 = 1316_8$, padding the left-most group $1 \rightarrow 001$. Padding on the *left* never changes the value (leading zeros are free), whereas padding on the right would multiply by a power of the base — which is why grouping always starts at the units end.

1.8 Remark — $\text{FFFFFF}_{16} = 77777777_8 = 2^{24} - 1$: All-Ones in Three Bases

Part (f) is the prettiest. The number is “all 1’s” in binary (24 of them), which is exactly $2^{24} - 1$. Because 24 is divisible by *both* 3 and 4, those 24 ones partition cleanly either way:

$$2^{24} - 1 = \underbrace{1 \cdots 1}_{24}_2 = \underbrace{F \cdots F}_6_{16} = \underbrace{7 \cdots 7}_8_8.$$

So the *same* number is a string of maximal digits in all three power-of-two bases — six *F*’s, eight 7’s, twenty-four 1’s. This also confirms the value: $\text{FFFFFF}_{16} = 16^6 - 1$ and $77777777_8 = 8^8 - 1$, and indeed $16^6 = (2^4)^6 = 2^{24} = (2^3)^8 = 8^8$, so both equal $2^{24} - 1 = 16,777,215$. (This is the “24-bit” all-ones value — three bytes of **FF**, as in a 24-bit RGB color **#FFFFFF**: pure white.)

1.9 Remark — Octal and Hex Are Each Other’s Cousins Through Binary

Parts (b) and (f) — the octal $\leftrightarrow$ hex conversions — never touched decimal. They went **octal** $\rightarrow$ **binary** $\rightarrow$ **hex** and **hex** $\rightarrow$ **binary** $\rightarrow$ **octal**, just by expanding to bits and re-cutting them into different-sized groups. There is no direct “octal digit $\rightarrow$ hex digit” table, because 3 and 4 don’t divide each other; but the least common multiple $\text{lcm}(3, 4) = 12$ does mean the two regroupings re-synchronise every 12 bits (= 4 octal digits = 3 hex digits). Binary is the universal translator that makes all three power-of-two bases freely interchangeable.